PART A – PEER CODE AUDIT CHECKLIST

#	Issue / Finding	Location	Severity	How the Refactored Code Addresses It
1	Global State – mutable global dataArray, currentCount, maxCapacity	Legacy lines 8–10	High	Encapsulated inside DynamicArray as private data, size, and capacity. No mutable global array state is used.
2	Memory Leak – resize without deleting old dataArray	Legacy lines 18–22	Critical	resize() allocates temp, copies the elements, calls delete[] data, then assigns data = temp and updates capacity.
3	Memory Leak – no final delete[] at program end	Legacy line 100	High	DynamicArray has a destructor that calls delete[] data automatically when records go out of scope.
4	Out-of-Bounds – printAll uses i <= currentCount	Legacy line 58	High	printAll() loops while i < size, so only valid elements are accessed.
5	Out-of-Bounds – remove_item_at lacks idx >= currentCount	Legacy line 31	Critical	removeItemAt() checks index < 0 and index >= size before accessing the array.
6	Inefficient Shift – redundant nested loop	Legacy lines 37–41	Medium	removeItemAt() uses one loop to shift remaining elements left; the redundant inner loop is removed.
7	Naming Standards – add_item vs findItem vs printAll	Multiple locations	Low	Public methods use consistent camelCase names: addItem, removeItemAt, findItem, printAll.
8	Hardcoded Sizing – fixed 3×3 matrix	Legacy lines 66–68	Medium	The supplied refactored code still demonstrates a fixed 3×3 matrix in processMatrix(). This audit item is therefore not fully generalized in the supplied refactoring.